Lounastutka

Lounastutka is a containerized full-stack app with:

- Frontend: React Router app
- Backend: Bun HTTP service
- Database: PostgreSQL

The repository includes two deployment modes:

- Local development with Docker Compose
- Production-style Docker Swarm stack with Traefik and monitoring services

Services Overview

Development (`compose.dev.yaml`):

- Frontend: `http://localhost:5173`
- Backend: `http://localhost:3001`
- PostgreSQL: `localhost:5432`

Swarm (`compose.yaml`):

- Traefik edge router: `http://localhost`
- Traefik dashboard: `http://localhost:8080`
- Grafana: `http://localhost:3000`
- Prometheus and Loki are internal by default

Documentation Map

- Installation and run instructions: Installation
- Frontend module notes: Frontend
- Backend module notes: Backend

Build Documentation

From repository root:

```
mkdocs build
```

Serve locally with live reload:

```
mkdocs serve
```

Installation

This guide reflects the current setup of the repository.

Prerequisites

- Docker Engine
- Docker Compose plugin (`docker compose`)
- Optional for docs: Python and MkDocs

Quick Start (Local Development)

Run from repository root:

```
docker compose -f compose.dev.yaml up
```

This starts:

- Frontend on `http://localhost:5173`
- Backend on `http://localhost:3001`
- PostgreSQL on `localhost:5432`

Stop services:

```
docker compose -f compose.dev.yaml down
```

Stop and remove volumes:

```
docker compose -f compose.dev.yaml down -v
```

Production-Style Swarm Deployment

Initialize Swarm (once):

```
docker swarm init
```

Build images:

```
docker build -t lounastutka-backend:latest ./Backend
docker build -t lounastutka-frontend:latest ./Frontend
```

Deploy stack:

```
docker stack deploy -c compose.yaml lounastutka
```

Useful checks:

```
docker stack services lounastutka
docker service ps lounastutka_app
docker service ps lounastutka_frontend
```

Swarm endpoints:

- App (via Traefik): `http://localhost`
- Traefik dashboard: `http://localhost:8080`
- Grafana: `http://localhost:3000`

Remove stack:

```
docker stack rm lounastutka
```

Notes

- In local development, frontend and backend source folders are bind-mounted into containers.
- Prometheus and Loki are included in Swarm mode and are internal by default.
- For real production, move credentials from Compose files into a secret manager.

Backend

Basic information regarding development and structure of the backend is described here. This is not supposed to be an indepth manual and explanation of the design decisions but a high overview of the initial idea for the structure and whats where.

Setup:

You need to have bun.js installed: bun.js installation

Install dependencies for the application backend:

```
[project_root]$ cd Backend/
[Backend]$ bun install
```

To run the server as standalone instance instead of docker container:

```
[Backend]$ bun run index.ts
```

For better testing it is recommended to run the whole setup with:

```
# To build and start the services:
[project_root]$ docker compose -f compose.dev.yaml up --build

# To stop them:
[project_root]$ docker compose -f compose.dev.yaml down

# And to remove the volumes as well:
[project_root]$ docker compose -f compose.dev.yaml down -v
```

Maintenance & Development

The backend is divided to the following structure:

```
[Backend]$ tree -I node_modules -I docs
.
├── bun.lock
├── controllers
│   ├── auth.controller.ts
│   ├── protected.controller.ts
│   ├── restaurant.controller.ts
│   └── types.ts
├── database
│   ├── helpers.ts
│   └── models.ts
├── Dockerfile
├── index.ts
├── middleware
│   ├── auth.middleware.ts
│   ├── auth.validator.ts
│   ├── error.middleware.ts
│   └── validator.ts
├── package.json
├── README.md
├── routes
│   ├── auth.routes.ts
│   ├── protected.routes.ts
│   └── restaurant.routes.ts
├── services
│   ├── auth.service.ts
│   ├── protected.service.ts
│   └── restaurant.service.ts
├── tsconfig.json
└── utils
    └── error.ts
```

Current dependencies can be found in the `package.json` file and package versions should be updated when vulneraibility patches come.

About the files, folders and their ideas:

index.ts: Entry to the backend application, defines the used features, e.g. express, cors etc.

utils/: Global utility methods and classes should lay here as they can be easily imported to the relevant source code files. E.g AppError.

routes/: routes that can be called to the backend defined here and which middleware are used, e.g. with this defined in index.ts: `app.use("/api/protected", protectedRoutes);` and in routes/protected.routes.ts `router.get("/hello-there", authenticateToken, helloThere);`.

controllers/: Here are defined the methods that handle calling appropriate services and acts as HTTP endpoint, the controllers deal with the request and response objects and if something goes wrong, calls the next middleware for errors or responds back to the caller with the

response object.

services/: These are called by the controllers and these methods handle the business logic of the backend application, calling queries to the database, potentially other services, e.g. the microservice etc.

database/: Acts as API to the database service, containing the database schema based typescript models and including the methods that make the database SQL queries via bun.js SQL library. It also has the connection initialization and its usage should be as easy as importing it to a service file and using e.g. `await db.findUserByID(...)`.

middleware/: This folder includes all of the custom middleware methods such as email & password verification and token verification and custom error handler for express.

API Documentation

The API documentation is built from the developer written `typedoc` and `JSDoc` style code comments and typedoc parsed method declarations. This is not the best way to do API documentation but considering the small codebase at this point it seemed to suffice, as the project grows it is beneficial to move towards e.g. OpenAPI style documentation format. The documentation index.html can be created with:

```
[Backend]$ bun run typedoc --entryPointStrategy Expand controllers/* middleware/*
```

, this should create the new html files under `Backend/docs/` folder.

NOTE: For better documentation, strong typing is recommended as typedoc can and will complain if there are 'any' types around.

Error handling

Currently there exists only the generic error handling code that shows the end-user generic error code rather than showing the full stack from backend as it could potentially contain protected information and in the end wouldn't help a normal user. This relies on the fact that when developing and using error messages, it is important to differentiate what types of errors the user should see and what should remain under the maintainers logs, e.g. 4xx codes vs 5xx codes.

NOTE: This project was created using `bun init` in bun v1.3.10. Bun is a fast all-in-one JavaScript runtime.

Microservice (Scraper Service)

This microservice is responsible for retrieving restaurant and lunch menu data for the Lounastutka application.

It acts as a dedicated data provider between the backend and external (or simulated) data sources.

Responsibilities

- Accept a restaurant URL from the backend
- Return structured restaurant data

Architecture Role

```
Frontend → Backend → Microservice → Backend → Database → Frontend
```

The microservice is **not exposed publicly** and is only accessible within the Docker network.

Overview

The service exposes a simple HTTP API that:

- Receives a restaurant URL
- Returns structured restaurant + menu data
- Provides either real or mock data depending on the source

Behavior

- **Aalef URL (`https://www.aalef.fi/#ravintolat`)**
- Attempts to scrape real menu data for the current day
- **All other URLs**

- Returns randomized mock data (for now)
- Locations are constrained within the Lappeenranta area

API

`POST /scrape`

Request:

```
{
  "url": "https://example.com"
}
```

Example response:

```
{
  "id": 1,
  "type": "restaurant",
  "position": [61.05, 28.19],
  "name": "Restaurant Name",
  "category": "Ravintola",
  "stars": 4.5,
  "reviews": 120,
  "address": "Example street 1",
  "description": "Description",
  "todayHours": "10:00-15:00",
  "lunchTime": "11:00-14:00",
  "priceLevel": "Lunch 12 EUR",
  "phone": "000000",
  "website": "https://example.com",
  "tags": ["lunch"],
  "todayMenu": ["Food 1", "Food 2"]
}
```

`GET /health`

Returns service health status:

```
{ "status": "ok" }
```

Running locally

```
docker compose up --build microservice
```

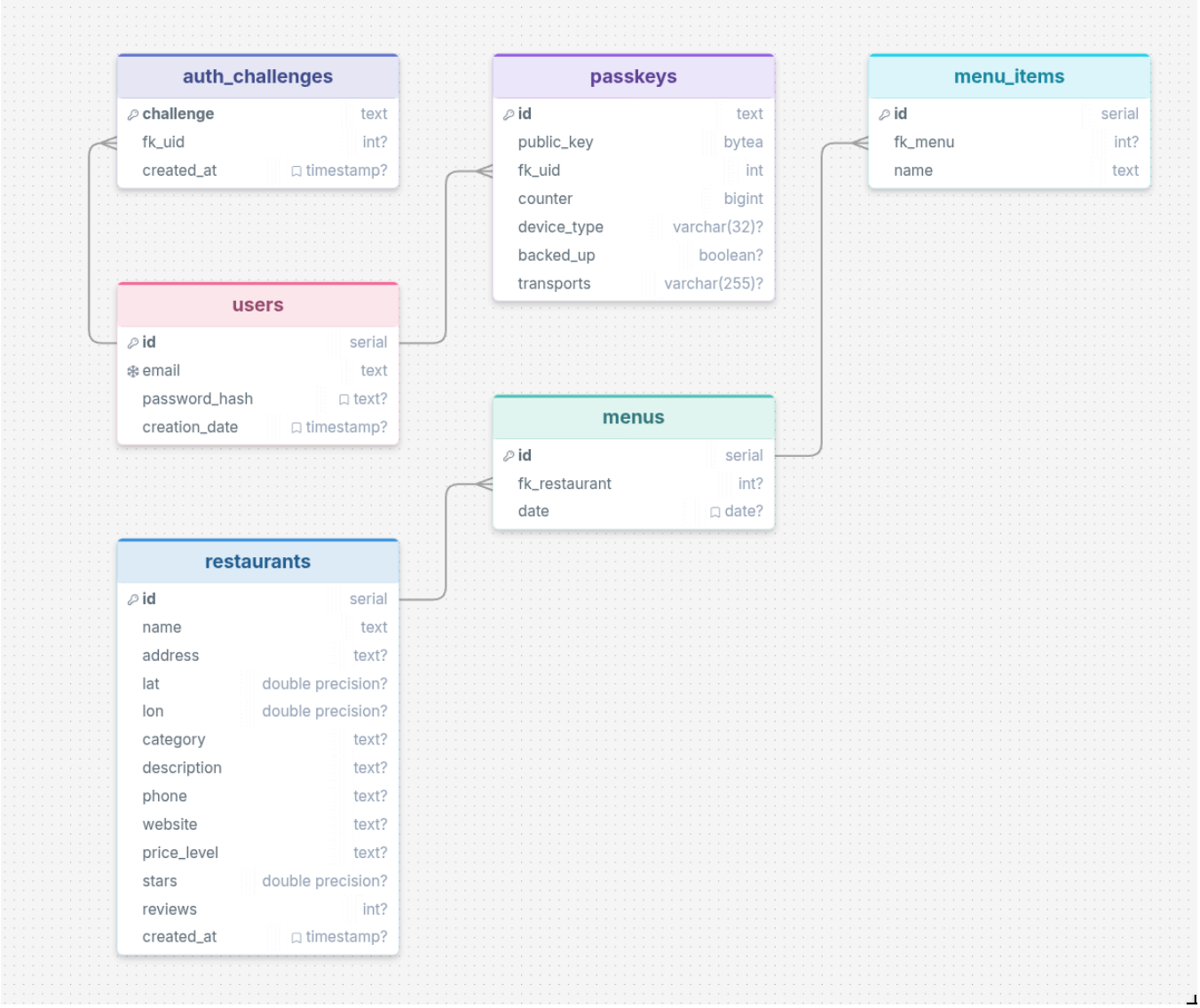
PostgreSQL DB

The project uses PostgresSQL relational database for storing relevant user and restaurant information. The database is run as a separate docker service and accessed by the backend docker service via the defined helpers in `Backend/database/`.

The .sql files under `Database/init/` are mounted to the docker service environment and executed during the first initialization of the database.

The user `lounastutka` is authorized to access the database and the environment variables should be set correctly for that user to have access from the backend. This setup is different for production and for local development and is described in `Environment` -section. Examples can be seen in .env-example and within the docker compose files.

Diagram built automatically with https://drawsql.app/ by importing the database schema.sql.


auth_challenges
challenge text
fk_uid int?
created_at timestamp?
passkeys
id text
public_key bytea
fk_uid int
counter bigint
device_type varchar(32)?
backed_up boolean?
transports varchar(255)?
menu_items
id serial
fk_menu int?
name text
users
id serial
email text
password_hash text?
creation_date timestamp?
menus
id serial
fk_restaurant int?
date date?
restaurants
id serial
name text
address text?
lat double precision?
lon double precision?
category text?
description text?
phone text?
website text?
price_level text?
stars double precision?
reviews int?
created_at timestamp?